Why Random Forests Work: Bias and Variance

Contents

1. Overview 2
2. Why a forest lowers the variance 3
3. Seeing it in classification 4
4. Seeing it in regression 6
5. Summary 7
6. Sources 7
7. Key terms 7

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	**9 Evaluate**	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Bias-variance trade-off; Random forest.
- **Compare with:** K-nearest neighbours; Bagging; Dropout.

1. Overview

Key point: A **random forest** (G-1611) takes fully grown trees, which have low bias and high **variance** (G-2074), and averages away most of the variance. The result has low bias and low variance, the combination single models rarely reach.

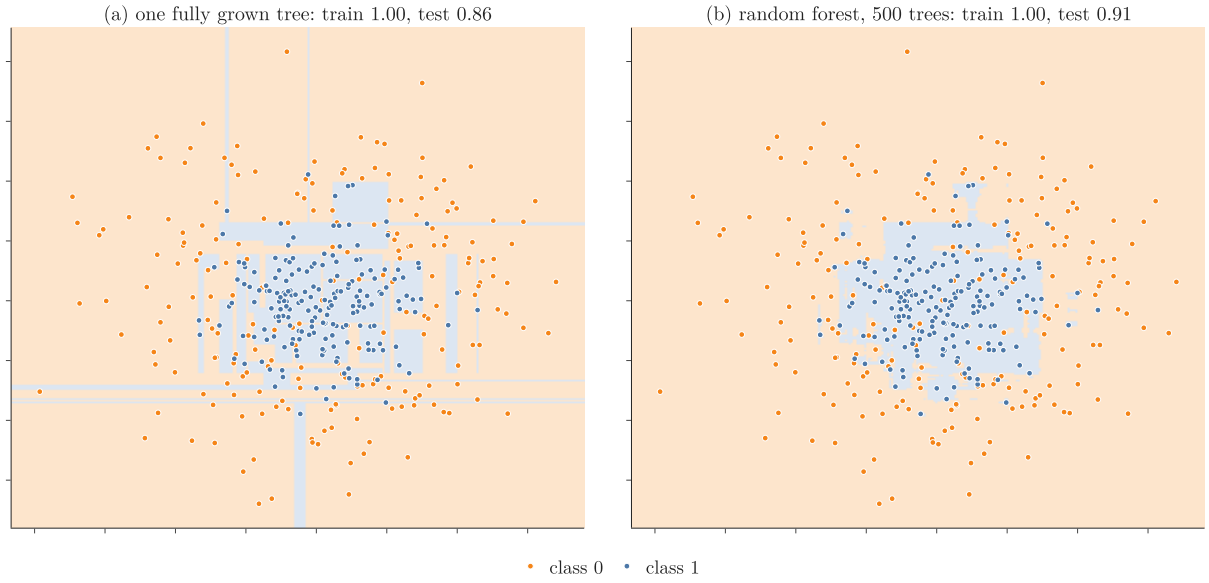


Figure 1: Concentric circles: (a) one fully grown tree draws boxes around single points; (b) a random forest of 500 trees draws one smooth inner region. Titles give the accuracy on the training and test sets

Figure 1 shows the whole story. One fully grown tree (a) carves out thin strips and small boxes for single points: it overfits. A forest of 500 such trees (b) keeps the shape of the data, a blue disc inside an orange ring, and drops most of the strips. Its test accuracy rises from 0.86 to 0.91.

This Note explains why, using the **bias-variance trade-off** (G-288), and shows the same effect on a regression problem. The Notebook (`ML-103-random-forest-bias-variance.ipynb`) computes every number; `images/figs.py` draws the figures.

2. Why a forest lowers the variance

> **Key point:** A **random forest** (G-1611) is bagging with fully grown trees: each tree keeps the bias low, and averaging many of them removes most of the variance.

Two words carry this Note (both explained in bias and variance):

- **Bias** (G-287): the error of a model that is too simple to follow the true pattern, such as a straight line through curved data. The model is wrong even on its own training data.
- **Variance** (G-2074): how much the fitted model changes when the training data changes. A model that hugs every training point fits them perfectly, then fits new data badly.

We want both low, but a single model usually trades one for the other. **Bagging** (G-251) escapes this by averaging low-bias, high-variance models (G-1132), each trained on its own random sample, a **bootstrap sample** (G-319), so a change in the data is spread across many models (see how bagging lowers variance). Think of many people guessing the number of sweets in a jar: each guess is off, some high and some low, but the average of the guesses lands close to the truth.

A random forest applies this to **fully grown decision trees** (G-561; no `max_depth`). Each tree fits its training data almost perfectly, so the forest starts with low bias; averaging hundreds of them keeps the bias low and cuts the variance. Figure 2 shows the recipe. The next two sections measure the effect. Averaging helps most when the trees differ from one another; the random choice of features at each split, which makes them differ more, is the subject of **node-level feature sampling** (G-1325).

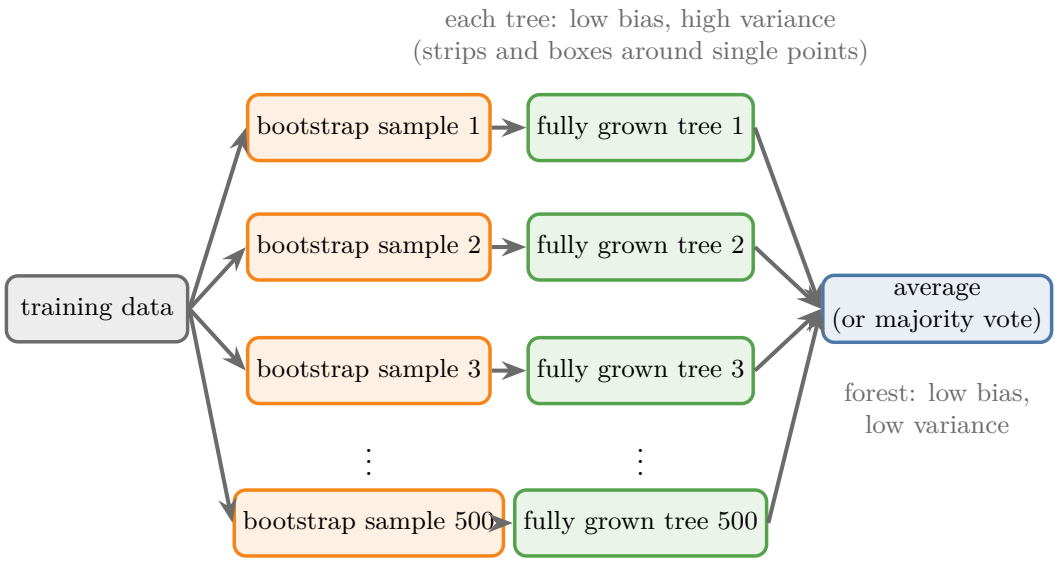


Figure 2: A random forest is bagging with fully grown trees. Each bootstrap sample grows one deep tree with low bias and high variance; averaging the 500 trees (or taking their majority vote) keeps the low bias and cancels most of the variance

3. Seeing it in classification

> **Key point:** On noisy concentric circles, one tree scores 0.86 on the test set and a forest of 500 trees 0.91.

The data is made with `make_circles` (see the data: two circles): 500 **observations** (G-1374; records, one row of the data table each), 2 **features** (G-772; input variables, one column each) and a **target** (G-1949; the output we predict) with two classes. The blue class (1) sits in a small disc inside the orange class (0), with plenty of noise, so the classes overlap. We train on 400 points and test on 100.

One fully grown tree (Figure 1a). Its surface has long thin strips and isolated boxes. Each exists because of a single point: a lone blue point among orange ones gets its own blue strip, although the region around it clearly belongs to orange.

- Training accuracy: **1.00**, every training point is classified correctly: low **bias** (G-287).
- Test accuracy: **0.86**. A new sample from the same pattern would produce different strips in different places: high **variance** (G-2074). This is **overfitting** (G-1429): the model fits the quirks of its training sample, not the pattern.

A random forest of 500 trees (Figure 1b). Most strips and boxes are gone. The surface captures the true nature of the data: a blue region inside an orange one.

- Training accuracy: still **1.00**.
- Test accuracy: **0.91**: the variance has dropped.

Variance means how much a model changes when the training data changes, so Figure 3 changes the data. It draws the circles eight times with the same recipe (draw 1 is the data of Figure 1) and trains both models on each draw. Watch the left panel: the tree's strips and boxes jump to new places with every draw. The forest's disc barely moves. The last frame overlays all eight surfaces: white marks the places where the draws disagree. Two draws of the tree disagree on 11% of the plane; two draws of the forest on 5%. Averaged over the eight draws, test accuracy is 0.80 for the tree and 0.83 for the forest.

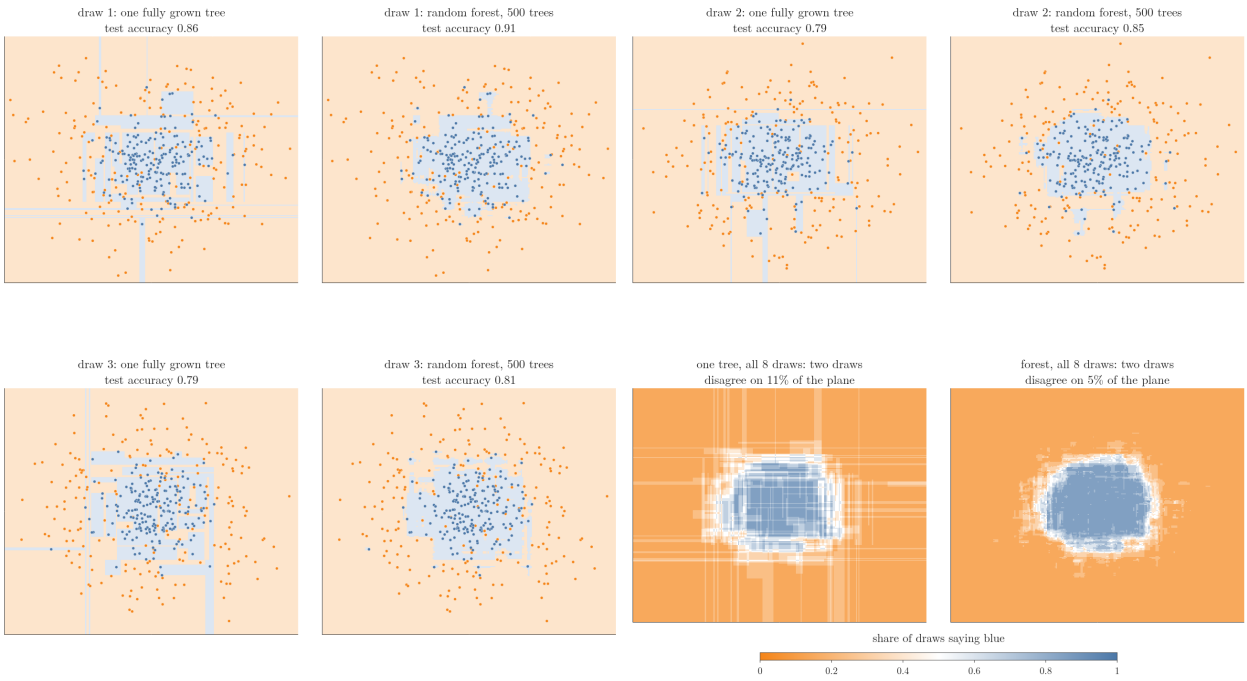


Figure 3: The circles data drawn eight times. Each frame: one fully grown tree (left) and a random forest of 500 trees (right) trained on that draw, with test accuracy. Last frame: the share of the eight surfaces that say blue; white means the draws disagree

Python: The two models.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier

tree = DecisionTreeClassifier(random_state=42)
tree.fit(X_train, y_train)
rf = RandomForestClassifier(n_estimators=500,
                            random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
rf.score(X_test, y_test)      # 0.91
```

`n_estimators` is the number of trees and `n_jobs=-1` trains them on every CPU core (both from BaggingClassifier in code). The surfaces are drawn on a grid of points, as in **decision surfaces** (G-560).

Extra: One might expect the forest to give up a little training accuracy in exchange for the lower variance. Here it does not. Each tree's **bootstrap sample** (G-319) holds about 63% of the training observations (ESL §7.11), and a fully grown tree gets every observation it saw right. So for any training observation, the trees that saw it are a majority, and they outvote the rest.

The Notebook checks this: every training observation was seen by at least 57% of the 500 trees, and those trees were always right on it. Raising the noise of `make_circles` up

to 1.0 leaves the forest's training accuracy at 1.00.

4. Seeing it in regression

Key point: On a noisy curve, one tree chases every point (test MSE 0.0192); a forest of 1,000 trees stays closer to the true pattern (test MSE 0.0140), exactly like **bagging** (G-251), since one feature leaves no features to sample.

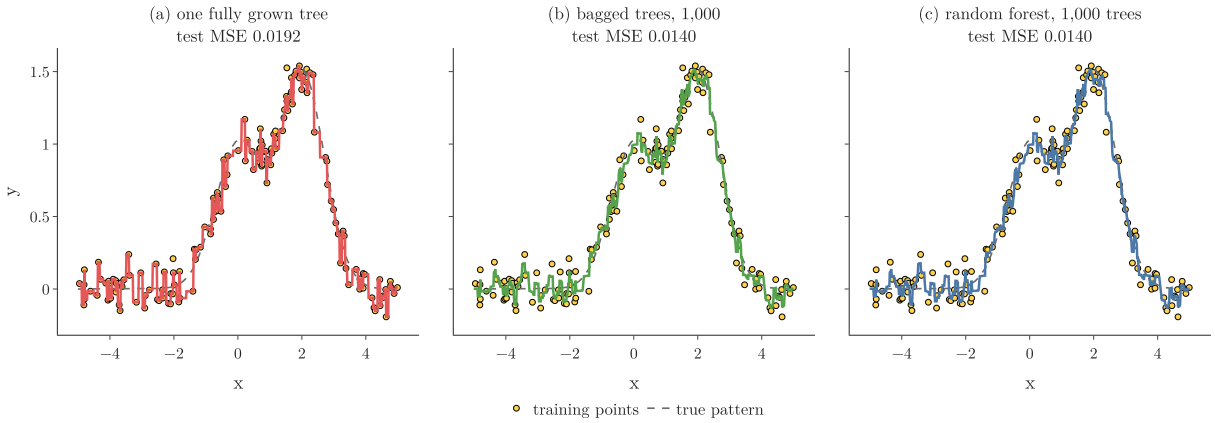


Figure 4: Two bumps plus noise: (a) one fully grown regression tree; (b) bagging with 1,000 fully grown trees; (c) a random forest of 1,000 trees. The dashed curve is the true pattern

The data is the two-bumps curve of seeing it on a curve, which compares one tree with bagging. Here we add the **random forest** (G-1611). We train on 150 points and test on 1,000; the dashed curve in Figure 4 is the true pattern.

- **(a) One fully grown tree** (red) passes through every training point: training error 0, test **mean squared error (MSE)** (G-1201; the average squared gap between prediction and truth, see **mean squared error** (G-1201)) **0.0192**.
- **(b) Bagging with 1,000 fully grown trees** (green) no longer reaches every outlier and stays closer to the dashed curve: test MSE **0.0140**.
- **(c) A random forest of 1,000 trees** (blue) draws almost the same curve, with the same test MSE. Its training MSE rises a little, from 0 to 0.0018, because each training point was missed by about a third of the trees (the Extra in section 3), and an average, unlike a majority vote, lets their slightly different values pull the prediction off the point; meanwhile its test MSE falls about 27% below the single tree's.

Panels (b) and (c) match because the data has a single feature. A random forest differs from bagging only in sampling features at each split (see **node-level feature sampling** (G-1325)); with one feature there is nothing to sample, and `RandomForestRegressor` uses every feature at every split by default anyway. The forest's extra gain shows only on data with many features.

5. Summary

	One fully grown tree	Random forest
Bias	low	low
Variance	high	low
Circles: train / test accuracy	1.00 / 0.86	1.00 / 0.91
Curve: train / test MSE	0.0000 / 0.0192	0.0018 / 0.0140

- We want low bias and low **variance** (G-2074), but single models usually trade one for the other, so a model that escapes the trade-off is worth having.
- Fully grown trees are low **bias** (G-287), high variance: they overfit, because they fit every training point (training accuracy 1.00) and draw strips around single noisy points that move with every new sample.
- A **random forest** (G-1611) averages many such trees, each trained on a different random sample, so noisy observations are spread out and their effect averages away.
- The result keeps the low bias and cuts the variance: smoother boundaries and curves, better scores on new data. This is why a random forest performs so well: each tree keeps the bias low, and the average removes most of the variance.

6. Sources

Built from

- CampusX, "How Random Forest Performs So Well? Bias Variance Trade-Off in Random Forest", YouTube, https://www.youtube.com/watch?v=jHgG4gjuFAk

Other references

- Hastie, T., Tibshirani, R. and Friedman, J. (2009). *The Elements of Statistical Learning*, 2nd ed. Springer. §7.11 (the bootstrap holds about 63.2% of distinct observations).

7. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Random forest (G-1611)	Bagging with decision trees as the base models: many trees, each trained on a random sample of the rows with a random choice of features at every split, vote (classification) or are averaged (regression), which lowers the variance of a single tree.

Term	Meaning
Variance (of data) (G-2074)	The average squared distance of the values from their mean; the square of the standard deviation; divide by n for a population (NumPy default) or $n-1$ for a sample (pandas default).
Bias-variance trade-off (G-288)	Lowering bias by adding complexity tends to raise variance, and the reverse.
Bias (of a model) (G-287)	Error from a model being too simple to capture the true relationship.
Bagging (bootstrap aggregation) (G-251)	Training many models on different random samples of the data and averaging them, so the result depends less on the particular sample (lower variance).
Low bias, high variance algorithm (G-1132)	An algorithm that fits its training data very well but changes a lot with the data, such as a fully grown tree; it overfits.
Bootstrap sample (G-319)	A sample of the same size as the data, drawn with replacement so some rows repeat and others are left out; each tree of a random forest gets its own, so the trees differ.
Decision tree (G-561)	A model that predicts by asking a chain of questions about the input columns: nested if-else conditions.
Node-level feature sampling (node-level column sampling) (G-1325)	Drawing a new random set of features before every split (random forest).
Observation (G-1374)	One record of a dataset: one row of the data table.
Feature (G-772)	One piece of information about each example that a model uses (e.g. a student's CGPA).
Target (G-1949)	The output column we predict, such as the class; also called the label.
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.
Decision surface (G-560)	A plot colouring every point of the input space by the class the model would predict there.
Mean squared error (MSE) (G-1201)	The average squared difference between actual and predicted values.